

```
In [ ]: import numpy as np
import pandas as pd
from statsmodels.formula.api import ols
from plotnine import ggplot, aes, geom_point, labs
from scipy.stats import linregress
```

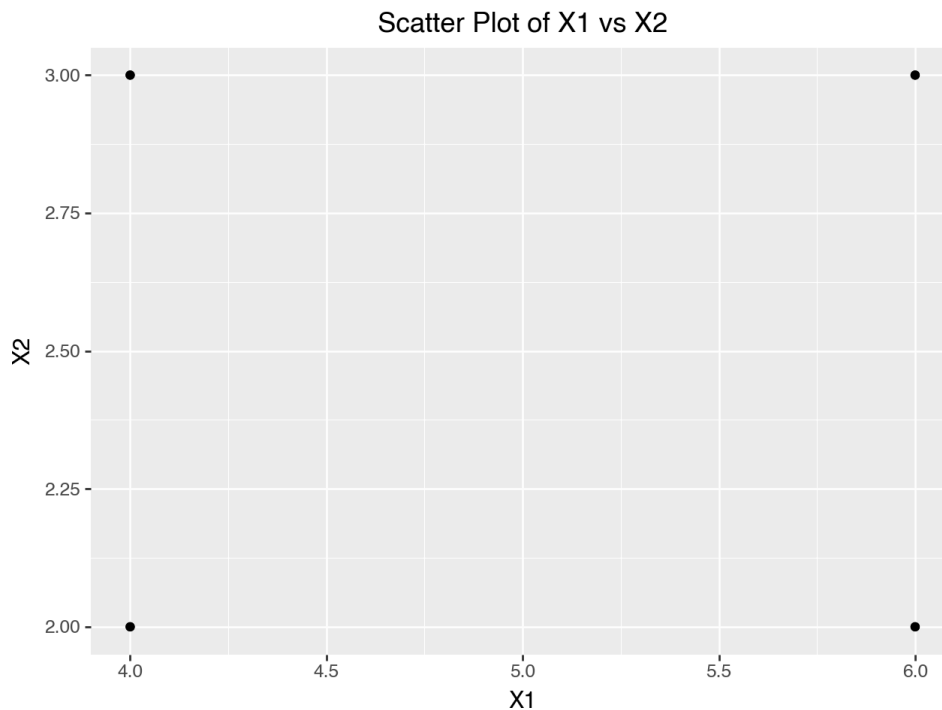
Multicollinearity Demo

```
In [ ]: ### multicollinearity

# Uncorrelated predictor variables
X1 = [4, 4, 4, 4, 6, 6, 6, 6]
X2 = [2, 2, 3, 3, 2, 2, 3, 3]
Y = [42, 39, 48, 51, 49, 53, 61, 60]
```

```
In [ ]: # Creating DataFrame
df = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'Y': Y
})

# Plot X1 vs X2
plot = (ggplot(df, aes(x='X1', y='X2')) +
        geom_point() +
        labs(x='X1', y='X2', title='Scatter Plot of X1 vs X2'))
plot
```



```
In [ ]: # Correlation between X1 and X2
correlation = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation}')
```

Correlation between X1 and X2: 0.0

```
In [ ]: # Linear model Y ~ X1 + X2
model1 = ols('Y ~ X1 + X2', data=df).fit()
model1.summary()
```

/opt/anaconda3/lib/python3.12/site-packages/scipy/stats/_axis_nan_policy.py:531: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=8

Out []:

OLS Regression Results

Dep. Variable:	Y	R-squared:	0.958
Model:	OLS	Adj. R-squared:	0.941
Method:	Least Squares	F-statistic:	57.06
Date:	Thu, 25 Sep 2025	Prob (F-statistic):	0.000361
Time:	19:03:15	Log-Likelihood:	-14.511
No. Observations:	8	AIC:	35.02
Df Residuals:	5	BIC:	35.26
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.3750	4.740	0.079	0.940	-11.811	12.561
X1	5.3750	0.664	8.097	0.000	3.669	7.081
X2	9.2500	1.328	6.968	0.001	5.837	12.663

Omnibus:	2.902	Durbin-Watson:	2.773
Prob(Omnibus):	0.234	Jarque-Bera (JB):	0.878
Skew:	-0.108	Prob(JB):	0.645
Kurtosis:	1.391	Cond. No.	42.1

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In []:

```
# Linear model Y ~ X1
model_X1 = ols('Y ~ X1', data=df).fit()
model_X1.summary()
```

/opt/anaconda3/lib/python3.12/site-packages/scipy/stats/_axis_nan_policy.py:531: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=8

Out[]:

OLS Regression Results						
Dep. Variable:	Y			R-squared:	0.550	
Model:	OLS			Adj. R-squared:	0.476	
Method:	Least Squares			F-statistic:	7.347	
Date:	Thu, 25 Sep 2025			Prob (F-statistic):	0.0351	
Time:	19:03:15			Log-Likelihood:	-23.995	
No. Observations:	8			AIC:	51.99	
Df Residuals:	6			BIC:	52.15	
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	23.5000	10.111	2.324	0.059	-1.242	48.242
X1	5.3750	1.983	2.711	0.035	0.523	10.227
Omnibus:	3.389	Durbin-Watson:		1.815		
Prob(Omnibus):	0.184	Jarque-Bera (JB):		0.935		
Skew:	-0.117	Prob(JB):		0.626		
Kurtosis:	1.341	Cond. No.		27.0		

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [ ]: # Linear model Y ~ X2
model_X2 = ols('Y ~ X2', data=df).fit()
model_X2.summary()
```

/opt/anaconda3/lib/python3.12/site-packages/scipy/stats/_axis_nan_policy.py:531: UserWarning: kurtosistest only valid for n>=20 ... continuing anyway, n=8

Out []:

OLS Regression Results						
Dep. Variable:		Y		R-squared:		0.408
Model:		OLS		Adj. R-squared:		0.309
Method:		Least Squares		F-statistic:		4.128
Date:		Thu, 25 Sep 2025		Prob (F-statistic):		0.0885
Time:		19:03:15		Log-Likelihood:		-25.100
No. Observations:		8		AIC:		54.20
Df Residuals:		6		BIC:		54.36
Df Model:		1				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
Intercept	27.2500	11.608	2.348	0.057	-1.153	55.653
X2	9.2500	4.553	2.032	0.088	-1.891	20.391
Omnibus:	4.296	Durbin-Watson:		0.359		
Prob(Omnibus):	0.117	Jarque-Bera (JB):		1.012		
Skew:	-0.008	Prob(JB):		0.603		
Kurtosis:	1.257	Cond. No.		14.9		

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In []:

```
# ANOVA for both models
from statsmodels.stats.anova import anova_lm

anova_results1 = anova_lm(model1)
anova_results1
```

Out []:

	df	sum_sq	mean_sq	F	PR(>F)
X1	1.0	231.125	231.125	65.567376	0.000466
X2	1.0	171.125	171.125	48.546099	0.000937
Residual	5.0	17.625	3.525	NaN	NaN

In []:

```
anova_results_X1 = anova_lm(model_X1)
anova_results_X1
```

Out []:

	df	sum_sq	mean_sq	F	PR(>F)
X1	1.0	231.125	231.125000	7.34702	0.035081
Residual	6.0	188.750	31.458333	NaN	NaN

In []:

```
anova_results_X2 = anova_lm(model_X2)
anova_results_X2
```

Out []:

	df	sum_sq	mean_sq	F	PR(>F)
X2	1.0	171.125	171.125000	4.127638	0.08846
Residual	6.0	248.750	41.458333	NaN	NaN

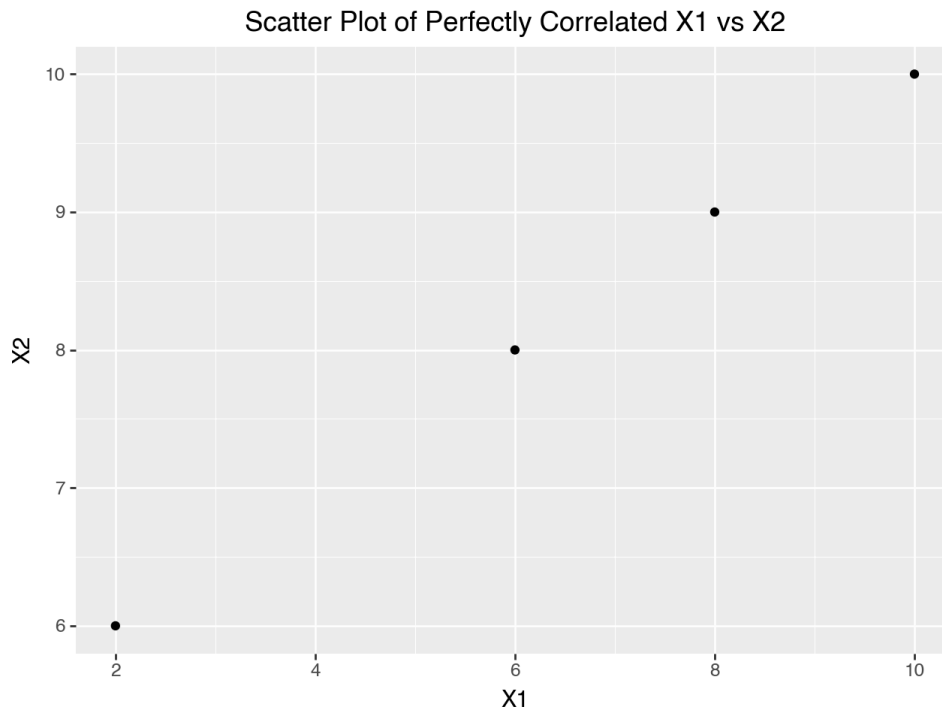
In []:

```
# Perfectly correlated case
X1 = [2, 8, 6, 10]
X2 = [6, 9, 8, 10]
Y = [23, 83, 63, 103]
```

```
df_perfect = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'Y': Y
})

# Plot X1 vs X2
plot_perfect = (ggplot(df_perfect, aes(x='X1', y='X2')) +
    geom_point() +
    labs(x='X1', y='X2', title='Scatter Plot of Perfectly Correlated X1 vs X2'))

plot_perfect
```



```
In [ ]: # Correlation between X1 and X2
correlation_perfect = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation_perfect}')
```

Correlation between X1 and X2: 1.0

```
In [ ]: X = np.column_stack([np.ones(len(X1)), X1, X2])
X
```

```
Out [ ]: array([[ 1.,  2.,  6.],
                [ 1.,  8.,  9.],
                [ 1.,  6.,  8.],
                [ 1., 10., 10.]])
```

```
In [ ]: # Matrix operations

XtX = np.dot(X.T, X)
print(f'XtX Matrix:\n{XtX}')
```

XtX Matrix:

```
[[ 4. 26. 33.]
 [26. 204. 232.]
 [33. 232. 281.]]
```

```
In [ ]: # Eigenvalues
eigenvalues = np.linalg.eigvals(XtX)
print(f'Eigenvalues:\n{eigenvalues}') #2nd eigen val very close to zero !!!!
#numerically near singular
#this is going to be an issue for us:
```

Eigenvalues:

```
[4.81365468e+02 7.42814262e-17 7.63453185e+00]
```

```
In [ ]: import scipy.linalg as spla
```

```
# Attempt to invert the matrix
XtX_inv = spla.solve(XtX, np.eye(3))
print(f'Inverse of XtX:\n{XtX_inv}')

#it blows up and gives you a warning on the "Condition Number" of the matrix
```

```
Inverse of XtX:
[[-4.56937087e+15 -4.56937087e+14  9.13874175e+14]
 [-4.56937087e+14 -4.56937087e+13  9.13874175e+13]
 [ 9.13874175e+14  9.13874175e+13 -1.82774835e+14]]
```

```
/var/folders/xj/rb62l69s6p1d_vb461s2vgcm0000gn/T/ipykernel_97723/3933071815.py:4: LinAlgWarning: Ill-conditioned matrix (rcond=3.08324e-19): result may not be accurate.
```

```
In [ ]: #the warning is basically telling you the result is nonsense. Sanity check!
```

```
In [ ]: XtX_inv @ XtX # this is not the identity.... cry
```

```
Out[ ]: array([[ 2.75    , -2.    , 16.75    ],
               [ 0.21875,  6.75    ,  4.46875],
               [-0.25    , -2.    , -4.25    ]])
```

```
In [ ]: #If you use np.linalg.inv ... it doesn't do the float calcs exactly and will actually let you invert a singular matrix
np.linalg.inv(XtX)
```

```
# Interesting discussion of the issue: https://github.com/numpy/numpy/issues/2074
```

```
Out[ ]: array([[-4.56937087e+15, -4.56937087e+14,  9.13874175e+14],
               [-4.56937087e+14, -4.56937087e+13,  9.13874175e+13],
               [ 9.13874175e+14,  9.13874175e+13, -1.82774835e+14]])
```

Variance Inflation Factors - VIFs

```
In [ ]: # Variance Inflation - generating new data
np.random.seed(0)
X1 = np.random.uniform(25, 50, 100)
X2 = np.random.normal(10, 3, 100)
eps = np.random.normal(0, 4, 100)

df_vif = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'Y': 7 + 3 * X1 + 5 * X2 + eps
})

# Correlation between X1 and X2
correlation_X1_X2 = np.corrcoef(X1, X2)[0, 1]
print(f'Correlation between X1 and X2: {correlation_X1_X2}')
```

Correlation between X1 and X2: -0.01824782721587361

```
In [ ]: # Linear model with X1 and X2
mod1 = ols('Y ~ X1 + X2', data=df_vif).fit()
mod1.summary()
```

Out []:

OLS Regression Results

Dep. Variable:	Y	R-squared:	0.978
Model:	OLS	Adj. R-squared:	0.978
Method:	Least Squares	F-statistic:	2193.
Date:	Thu, 25 Sep 2025	Prob (F-statistic):	1.82e-81
Time:	19:03:16	Log-Likelihood:	-277.67
No. Observations:	100	AIC:	561.3
Df Residuals:	97	BIC:	569.2
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.9068	2.505	2.358	0.020	0.934	10.880
X1	2.9676	0.055	54.178	0.000	2.859	3.076
X2	5.1591	0.132	39.062	0.000	4.897	5.421

Omnibus:	0.196	Durbin-Watson:	2.109
Prob(Omnibus):	0.907	Jarque-Bera (JB):	0.026
Skew:	0.032	Prob(JB):	0.987
Kurtosis:	3.046	Cond. No.	247.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [ ]: vif_model_X1o = ols('X1 ~ X2', data=df_vif).fit()
r2j_X1o = vif_model_X1o.rsquared
vif_j_x1 = 1/(1-r2j_X1o)
print(f'VIF for X1: {vif_j_x1}')
```

VIF for X1: 1.000333094112843

VIF Close to 1 indicates not very much multicollinearity between X1 and other predictors.

On the other hand...

```
In [ ]: # Creating a nearly perfectly collinear variable X3
df_vif['X3'] = 2 * df_vif['X1'] + 3 * df_vif['X2'] + np.random.normal(0, 0.01, 100)

correlation_X1_X3 = np.corrcoef(df_vif['X1'], df_vif['X3'])[0, 1]
correlation_X2_X3 = np.corrcoef(df_vif['X2'], df_vif['X3'])[0, 1]
correlation_comb = np.corrcoef(2 * df_vif['X1'] + 3 * df_vif['X2'], df_vif['X3'])[0, 1]

print(f'Correlation between X1 and X3: {correlation_X1_X3}')
print(f'Correlation between X2 and X3: {correlation_X2_X3}')
print(f'Correlation between (2*X1 + 3*X2) and X3: {correlation_comb}')
```

Correlation between X1 and X3: 0.8464409676819135
Correlation between X2 and X3: 0.5169479332790725
Correlation between (2*X1 + 3*X2) and X3: 0.9999998570072173

```
In [ ]: # Linear model with X1, X2, and X3
mod2 = ols('Y ~ X1 + X2 + X3', data=df_vif).fit()
mod2.summary()
```

Out []:

OLS Regression Results

Dep. Variable:	Y	R-squared:	0.979
Model:	OLS	Adj. R-squared:	0.978
Method:	Least Squares	F-statistic:	1469.
Date:	Thu, 25 Sep 2025	Prob (F-statistic):	4.67e-80
Time:	19:03:16	Log-Likelihood:	-276.92
No. Observations:	100	AIC:	561.8
Df Residuals:	96	BIC:	572.3
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.8079	2.501	2.322	0.022	0.844	10.772
X1	-102.9961	87.588	-1.176	0.243	-276.858	70.866
X2	-153.7766	131.375	-1.171	0.245	-414.553	107.000
X3	52.9822	43.795	1.210	0.229	-33.949	139.914
Omnibus:	0.280	Durbin-Watson:	2.107			
Prob(Omnibus):	0.870	Jarque-Bera (JB):	0.044			
Skew:	0.020	Prob(JB):	0.978			
Kurtosis:	3.094	Cond. No.	4.73e+04			

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 4.73e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [ ]: # VIF calculation for X1
vif_model_X1 = ols('X1 ~ X2 + X3', data=df_vif).fit()
r2j_X1 = vif_model_X1.rsquared
vif_j_X1 = 1 / (1 - r2j_X1)
print(f'VIF for X1: {vif_j_X1}')
```

VIF for X1: 2570128.137094504

```
In [ ]: # VIF calculation for X3
vif_model_X2 = ols('X2 ~ X1 + X3', data=df_vif).fit()
r2j_X2 = vif_model_X2.rsquared
vif_j_X2 = 1 / (1 - r2j_X2)
print(f'VIF for X2: {vif_j_X2}')
```

VIF for X2: 994491.2150962795

```
In [ ]: # VIF calculation for X3
vif_model_X3 = ols('X3 ~ X1 + X2', data=df_vif).fit()
r2j_X3 = vif_model_X3.rsquared
vif_j_X3 = 1 / (1 - r2j_X3)
print(f'VIF for X3: {vif_j_X3}')
```

VIF for X3: 3506271.326686703

VIFs>>>10 indicate severe multicollinearity among predictors.

In []:

Exercise

For the dataset "multicollinearity.txt", perform the following:

1. Check the pairwise correlations of the predictors.
2. Compare the coefficient for each predictor in two cases: a. when regressing Y on that predictor only, and b. when regressing Y on all predictors. Do you see a major change in the coefficients? Their standard errors? What does this tell you about the presence of multicollinearity?
3. Calculate the VIFs for each predictor. Which VIFs are concerning? Comment on what you find.
4. Consider regressing `X4` onto the other predictors `X1` through `X3`. Save the residuals from this model and call them `res4`. Then regress Y onto `res4` and look at the coefficient. Compare this coefficient to the coefficients for `X4` you found in part 2. What do you notice?
5. Explain geometrically what we are doing in #4. Are we surprised by the results?

part 1

```
In [ ]: data = pd.read_csv("../data/multicollinearity.txt", delimiter="\t")
data.head(5)
```

```
Out [ ]:
```

	y	x1	x2	x3	x4
0	1.860	0.374	0.846	1.951	1.325
1	9.747	2.184	3.754	5.099	5.372
2	12.707	2.164	3.635	2.505	2.582
3	12.822	5.595	4.805	3.653	3.399
4	5.160	5.330	2.611	1.873	1.474

```
In [ ]: # pairwise correlation
correlation_matrix = data[['x1', 'x2', 'x3', 'x4']].corr()
correlation_matrix
```

```
Out [ ]:
```

	x1	x2	x3	x4
x1	1.000000	0.999811	0.999495	0.998843
x2	0.999811	1.000000	0.999671	0.999013
x3	0.999495	0.999671	1.000000	0.999306
x4	0.998843	0.999013	0.999306	1.000000

part 2

```
In [ ]: # Regress y onto x1
model_x1 = ols('y ~ x1', data=data).fit()
model_x1.params
```

```
Out [ ]:
```

Intercept	1.725665
x1	2.124962

dtype: float64

```
In [ ]: # Regress y onto x2
model_x2 = ols('y ~ x2', data=data).fit()
model_x2.params
```

```
Out [ ]:
```

Intercept	1.370597
x2	2.361322

dtype: float64

```
In [ ]: # Regress y onto x3
model_x3 = ols('y ~ x3', data=data).fit()
model_x3.params
```

```
Out [ ]:
```

Intercept	1.083137
x3	2.949185

dtype: float64

```
In [ ]: # Regress y onto x4
model_x4 = ols('y ~ x4', data=data).fit()
model_x4.params
```

```
Out[ ]: Intercept    0.896368
x4                4.200249
dtype: float64
```

```
In [ ]: # Regress y onto all predictors
model_all = ols('y ~ x1 + x2 + x3 + x4', data=data).fit()
model_all.params
```

```
Out[ ]: Intercept    1.037358
x1          -0.167540
x2           1.136449
x3           0.978820
x4           1.117531
dtype: float64
```

The coefficients for the params are much lower when regressing y onto all predictors versus regressing y onto one at a time.

```
In [ ]: # SE for x1
model_x1.bse
```

```
Out[ ]: Intercept    0.659300
x1           0.005688
dtype: float64
```

```
In [ ]: # SE for x2
model_x2.bse
```

```
Out[ ]: Intercept    0.554270
x2           0.005307
dtype: float64
```

```
In [ ]: # SE for x3
model_x3.bse
```

```
Out[ ]: Intercept    0.524205
x3           0.006262
dtype: float64
```

```
In [ ]: # SE for x4
model_x4.bse
```

```
Out[ ]: Intercept    0.683196
x4           0.011617
dtype: float64
```

```
In [ ]: # SE for all
model_all.bse
```

```
Out[ ]: Intercept    0.461542
x1           0.204055
x2           0.280617
x3           0.255575
x4           0.210430
dtype: float64
```

The SE's on the model where Y is regressed to all predictors are much higher than the individual ones

part 3

```
In [ ]: # VIF calculation for X1
vif_model_X1 = ols('x1 ~ x2 + x3 + x4', data=data).fit()
r2j_X1 = vif_model_X1.rsquared
vif_j_X1 = 1 / (1 - r2j_X1)
print(f'VIF for X1: {vif_j_X1}')
```

VIF for X1: 2653.8686128804293

```
In [ ]: # VIF calculation for X2
vif_model_X2 = ols('x2 ~ x1 + x3 + x4', data=data).fit()
r2j_X2 = vif_model_X2.rsquared
vif_j_X2 = 1 / (1 - r2j_X2)
print(f'VIF for X2: {vif_j_X2}')
```

VIF for X2: 2653.8686128804293

```
In [ ]: # VIF calculation for X3
vif_model_X3 = ols('x3 ~ x1 + x2 + x4', data=data).fit()
r2j_X3 = vif_model_X3.rsquared
vif_j_X3 = 1 / (1 - r2j_X3)
print(f'VIF for X3: {vif_j_X3}')
```

VIF for X3: 2162.4583519023327

```
In [ ]: # VIF calculation for X4
vif_model_X4 = ols('x4 ~ x1 + x2 + x3', data=data).fit()
r2j_X4 = vif_model_X4.rsquared
vif_j_X4 = 1 / (1 - r2j_X4)
print(f'VIF for X4: {vif_j_X4}')
```

VIF for X4: 722.2889185367274

The VIFs for all of these are extremely concerning as they are each much higher than 10

part 4

```
In [ ]: res4 = vif_model_X4.resid
res4
```

```
Out[ ]: 0      -0.089034
1       1.792657
2       0.724258
3       0.727658
4       0.033488
...
195     -1.007361
196     -0.034044
197     -1.243194
198     -2.572986
199      1.140979
Length: 200, dtype: float64
```

```
In [ ]: model_res4 = ols('y ~ res4', data=data).fit()
model_res4.summary()
```

Out[]:

OLS Regression Results						
Dep. Variable:	y			R-squared:	0.000	
Model:	OLS			Adj. R-squared:	-0.005	
Method:	Least Squares			F-statistic:	0.01938	
Date:	Thu, 25 Sep 2025			Prob (F-statistic):	0.889	
Time:	19:03:28			Log-Likelihood:	-1245.7	
No. Observations:	200			AIC:	2495.	
Df Residuals:	198			BIC:	2502.	
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	215.3599	8.716	24.707	0.000	198.171	232.549
res4	1.1175	8.028	0.139	0.889	-14.714	16.949
Omnibus:	91.438	Durbin-Watson:	0.003			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	11.967			
Skew:	-0.004	Prob(JB):	0.00252			
Kurtosis:	1.802	Cond. No.	1.09			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The coefficient is comparable to the coefficients when we regressed Y onto all the predictors.

part 5

Geometrically, res4 is orthogonal to the space of predictors and Y is projecting onto that